

Trabajo Práctico Integrador — Etapa 2

Implementación y demostración · Comunicación de Datos 2025 · Comisión S33

00 Integrantes del grupo

Santoro, José Joaquín
Legajo 33647

Aramburu, Tomás
Legajo 34650

Pagotto, Ignacio
Legajo 33605

Leguizamón, Nicolás
Legajo 33592

01 Resumen de la entrega

A partir de la arquitectura y los mockups definidos en la Etapa 1, se desarrolló la aplicación completa: un front-end en React que consume una API REST construida con FastAPI. El back-end implementa el muestreo y la cuantización de imágenes (Pillow) y los algoritmos de codificación Huffman y Shannon-Fano (biblioteca estándar de Python). Esta entrega incluye el código fuente completo, el instrumento de ejecución para que la cátedra pueda probar el desarrollo, y el enlace al video demostrativo.

02 Funcionalidades implementadas

Propuesta 3 — Digitalización de imágenes

- ✓ Carga de imágenes en alta resolución desde el navegador.
- ✓ Muestreo con distintos niveles de resolución (100×100, 500×500, 1000×1000), manteniendo la relación de aspecto.
- ✓ Reducción de profundidad de color a 1, 8 o 24 bits por canal.
- ✓ Comparación visual lado a lado: imagen original vs. digitalizada.
- ✓ Compresión del resultado (JPEG con calidad reducida) e indicador de tamaño de archivo (original vs. digitalizado, con % de reducción).

Propuesta 5 — Codificación de datos

- ✓ Carga de texto por entrada directa o archivo .txt.
- ✓ Cálculo de frecuencias y construcción del árbol y la tabla de códigos, tanto para Huffman como para Shannon-Fano.
- ✓ Codificación del mensaje a cadena binaria y decodificación inversa a partir de la tabla de códigos.
- ✓ Comparación de resultados entre ambos métodos: tasa de compresión, longitud promedio de código y entropía de la fuente.
- ✓ Gráfico de barras de frecuencias por símbolo, junto con la visualización del árbol binario de códigos (SVG, sin librerías externas).

03 Código fuente

El código fuente completo se encuentra organizado en dos proyectos independientes dentro del repositorio, separando claramente front-end y back-end, y dentro de cada componente, separando JS/JSX de CSS:

```
backend/app/main.py · routers/imagen.py · routers/codificar.py
backend/app/services/imagen_service.py · huffman.py · shannon_fano.py · arbol.py
backend/app/models/schemas.py
frontend/src/App.jsx + App.css
frontend/src/components/ImagenDigitalizador/*.jsx + *.css
frontend/src/components/Codificador/*.jsx + *.css
frontend/src/api/client.js
frontend/src/styles/index.css + shared.css
```

Repositorio público de GitHub: **[COMPLETAR CON EL LINK AL REPOSITORIO]**

04 Instrumento de ejecución

Para que la cátedra pueda testear el desarrollo de forma local, el repositorio incluye un **README.md** con las instrucciones detalladas. En resumen:

Back-end (FastAPI):

```
cd backend
python -m venv venv
venv\Scripts\activate (Windows) / source venv/bin/activate (Linux/Mac)
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Front-end (React + Vite):

```
cd frontend
npm install
npm run dev
```

El front-end queda disponible en <http://localhost:5173> y el back-end (con documentación interactiva Swagger) en <http://localhost:8000/docs>. Requisitos: Python 3.10+ y Node.js 18+.

05 Evidencia de funcionamiento

Pruebas realizadas sobre la API durante el desarrollo, con el texto "comunicacion de datos utn la plata" (35 caracteres, 272 bits sin comprimir):

Método	Bits codificados	Tasa de compresión	Long. promedio	Entropía
Huffman	124	54.41 %	3.647 bits	3.61 bits
Shannon-Fano	129	54.04 %	3.676 bits	3.61 bits

Se verificó además el ciclo completo de codificación + decodificación (round-trip), obteniendo exactamente el texto original a partir de la cadena binaria y la tabla de códigos. Para la digitalización de imágenes se probó una imagen de 800×600 px (2.7 KB): muestreada a 100×100 con 8 bits por canal y compresión activada, el resultado ocupó 1.3 KB.

06 Video demostrativo

Enlace al video (máximo 6 minutos) que muestra el uso de las funcionalidades desarrolladas: **[COMPLETAR CON EL LINK AL VIDEO]**